💡 Ключевые решения

1. DataValidator — ОДИН класс (не 6!)

Все данные — это числа разного вида. Всё сводится к одной логике:

```
скаляр  → float32, complex64                    → np.atleast_1d → ravel
вектор  → float32[n], complex64[n]              → ravel
матрица → float32[n,m], complex64[n,m]           → ravel
```

Три метрики покрывают все случаи:
- `"max_rel"` → `max(|a-r|) / max(|r|) < tol` — сигналы, спектр, статистика
- `"abs"` → `max(|a-r|) < tol` — частоты в Гц
- `"rmse"` → `rms(|a-r|) / rms(|r|) < tol` — шумные данные

2. AntennaProcessorTest — ОДИН объект, не пересоздаётся

Это важно понять! Объект создаётся ОДИН раз и живёт весь тест:

```
proc = ctx.create_processor_test(cfg)   ← создаётся один раз

proc.step_0_prepare_input(d_S, d_W)    ← пишет d_S_, d_W_ в state объекта
proc.step_2_gemm()                      ← читает d_S_/d_W_, пишет d_X_ в VRAM
proc.step_4_window_fft()               ← читает d_X_, пишет d_spectrum_ в VRAM
proc.step_6_1_one_max_parabola()       ← читает d_spectrum_/d_magnitudes_
```

Порядок вызова важен — не потому что объект пересоздаётся, а потому что:
- `step_2_gemm()` нужен `d_S_` (установлен в step_0) → без step_0 = nullptr
- `step_4_window_fft()` нужен `d_X_` (заполнен в step_2) → без step_2 = мусор
- `step_6_*` нужен `d_spectrum_` (заполнен в step_4) → без step_4 = мусор

`PipelineStepValidator` — это координатор (Facade + Template Method):
- вызывает `proc.step_N()` в правильном порядке
- читает результаты → сравнивает с NumPy
- объект `proc` не пересоздаётся!

🔬 Типы данных — везде float32! (проверено по коду)

Буфер / Структура	C++ тип	NumPy тип	Примечание
`d_S`	`complex<float>`	`np.complex64`	входной сигнал
`d_W`	`complex<float>`	`np.complex64`	весовая матрица
`d_X` (после GEMM)	`complex<float>`	`np.complex64`	X = W×S
`d_spectrum` (после FFT)	`complex<float>`	`np.complex64`	комплексный спектр
`d_magnitudes`	`float`	`np.float32`	`|spectrum|`
`StatisticsResult.mean`	`complex<float>`	`np.complex64`	комплексное среднее
`StatisticsResult.variance`	`float`	`np.float32`	дисперсия от `|z|`
`StatisticsResult.std_dev`	`float`	`np.float32`	σ(`|z|`)
`StatisticsResult.mean_magnitude`	`float`	`np.float32`	E[`|z|`]
`OneMaxParabolaLite.refined_freq_hz`	`float`	`np.float32`	частота пика
`MinMaxResult.dynamic_range_dB`	`float`	`np.float32`	динамический диапазон

⚠️ **STEP 1 и STEP 3 — вход комплексный, выход mixed:**
- Входные данные: `complex<float>` (d_S или d_X)
- `mean` возвращается как `complex<float>` (комплексное среднее)
- `variance/std_dev/mean_magnitude` — вычисляются от магнитуд `|z|`, возвращаются как `float`

⚠️ **STEP 5 — вход float:**
- Входные данные: `d_magnitudes` (`float`) — уже взяты магнитуды после FFT

NumPy reference — использовать `np.float32` / `np.complex64`, не float64!
Допустимо cast к float64 только внутри DataValidator для вычислений (точность метрики).

📊 5 ВАРИАНТОВ СИГНАЛОВ

```
class SignalVariant(Enum):
    """Выбор сценария перед стартом теста.

    Information Expert (GRASP): знает что нужно для каждого варианта.
    """
    V1_CW_CLEAN          = 1   # CW без шума,  W = Identity
    V2_CW_NOISE          = 2   # CW + AWGN,    W = Identity
    V3_CW_PHASE_DELAY    = 3   # CW,  фазовая задержка, W = delay_and_sum
    V4_CW_PHASE_NOISE    = 4   # CW + AWGN,   W = delay_and_sum
    V5_FROM_FILE         = 5   # Загрузка из файла (заглушка)
```

Вариант	Сигнал	Весовая матрица W	Что тестируем	Интерес
V1	CW, noise=OFF	W = I (Identity)	GEMM тривиален: X≡S, проверяем базовый pipeline	Простейший
V2	CW, SNR=20дБ	W = I	то же + шум не разрушает результат	Шум
V3	CW, noise=OFF	W = delay_and_sum	Нетривиальный GEMM: X≠S, формирование луча	Алгоритм
V4	CW, SNR=20дБ	W = delay_and_sum	Полный реальный сценарий	Production
V5	Из файла (CPU→GPU)	Из файла или I	Тест загрузки данных	Будущее

Для V3/V4: WeightGenerator.delay_and_sum(tau_step=100мкс, f0=2МГц)

🗺️ ПОЛНАЯ ДИАГРАММА PIPELINE ШАГОВ

```
╔═══════════════════════════════════════════════════════════════════════════╗
║           PYTHON TEST PIPELINE — strategies                               ╒
║           Стиль: ООП / SOLID / GRASP / GoF                                ╒
╠═══════════════════════════════════════════════════════════════════════════╣
╒                                                                           ╒
╒  ┌─ SETUP BLOCK ──────────────────────────────────────────────────────┐  ╒
╒  │                                                                    │  ╒
╒  │  [A] GPU SignalGenerator → d_S + S_ref                             │  ╒
╒  │      Выбор: SignalVariant.V1 / V2 / V3 / V4 / V5                   │  ╒
╒  │      V1: gw.CwGenerator(fs, f0, n_samples, n_ant, noise=False)     │  ╒
╒  │      V2: gw.CwGenerator(fs, f0, n_samples, n_ant, snr_db=20)       │  ╒
╒  │      V3: gw.CwGenerator(...)  + FarrowDelay для задержек           │  ╒
╒  │      V4: gw.CwGenerator(...)  + FarrowDelay + шум                  │  ╒
╒  │      V5: load_from_file(path) → hipMemcpyH2D  [ЗАГЛУШКА]           │  ╒
╒  │                                                                    │  ╒
╒  │      → d_S : VRAM [n_ant × n_samples] complex64  (GPU)             │  ╒
╒  │      → S_ref: CPU  [n_ant, n_samples] np.complex64                 │  ╒
╒  │                                                                    │  ╒
╒  │  [B] WeightGenerator → d_W + W_ref                                 │  ╒
╒  │      V1/V2: W = np.eye(n_ant, dtype=np.complex64)   (Identity)     │  ╒
╒  │      V3/V4: W = WeightGenerator.delay_and_sum(...)                 │  ╒
╒  │      V5:    W = Identity (по умолчанию)                            │  ╒
╒  │                                                                    │  ╒
╒  │      → d_W : VRAM [n_ant × n_ant] complex64                        │  ╒
╒  │      → W_ref: CPU  [n_ant, n_ant] np.complex64                     │  ╒
╒  │                                                                    │  ╒
╒  │  [C] NumpyReference(S_ref, W_ref, fs, f0, n_fft)                   │  ╒
╒  │      Вычисляет всё заранее: X_ref, spec_ref, stats, expected_bin   │  ╒
╒  └────────────────────────────────────────────────────────────────────┘  ╒
╒                              │                                            ╒
╒                              ▼                                            ╒
╒  ╔═══════════════════════════════════════════════════════════════════╗    ╒
╒  ╒  proc = AntennaProcessorTest(backend, cfg)   ← СОЗДАЁТСЯ ОДИН РАЗ  ╒    ╒
╒  ╚═══════════════════════════════╤═══════════════════════════════════╝    ╒
╒                                 │                                         ╒
╒  ┌─ STEP 0 ────────────────────▼──────────────────────────────────┐     ╒
╒  │  proc.step_0_prepare_input(d_S, d_W)                            │     ╒
╒  │  Записывает d_S_ и d_W_ в internal state объекта                │     ╒
╒  │                                                                 │     ╒
╒  │  ✅ CHECK-0: type(d_S) == complex64, shape == (n_ant, n_samples)  │     ╒
╒  │             type(d_W) == complex64, shape == (n_ant, n_ant)     │     ╒
╒  └─────────────────────────────────────────────────────────────────┘     ╒
╒                              │                                            ╒
╒       ┌──────────────────────┴──────────────────────┐                    ╒
╒       │ Stream debug1                               │ Stream main        ╒
╒       ▼                                             ▼                    ╒
╒  ┌─ STEP 1 ──────────────┐            ┌─ STEP 2 ──────────────────────┐ ╒
╒  │ step_1_debug_input()  │            │ step_2_gemm()                 │ ╒
╒  │                       │            │                               │ ╒
╒  │ Statistics(d_S)→ CPU  │            │ d_X = W × S → D2H → CPU       │ ╒
╒  │ Вход: complex64       │            │ Возвращает:                   │ ╒
╒  │ Выход: AntennaResult  │            │   List[complex<float>]        │ ╒
╒  │  .pre_input_stats[]:  │            │   → np.array(complex64)       │ ╒
╒  │   .mean  : complex64  │            │   shape: (n_ant × n_samples,) │ ╒
╒  │   .variance : float32 │            │   reshape → (n_ant, n_samples) │ ╒
╒  │   .std_dev  : float32 │            │                               │ ╒
╒  │   .mean_mag : float32 │            │ NumPy reference:              │ ╒
╒  │                       │            │   X_ref = W_ref @ S_ref       │ ╒
╒  │ NumPy reference:      │            │           (complex64 @ complex64)│ ╒
╒  │   mean = S_ref.mean(1)│            │                               │ ╒
╒  │   var  = |S_ref|.var  │            │ V1/V2 (W=I): X_ref == S_ref   │ ╒
╒  │   std  = |S_ref|.std  │            │ V3/V4 (W=d_s): X_ref ≠ S_ref  │ ╒
╒  │   mmag = |S_ref|.mean │            │                               │ ╒
╒  │                       │            │ ✅ CHECK-2:                   │ ╒
╒  │ ✅ CHECK-1a: mean     │            │   DataValidator(tol=1e-3,     │ ╒
╒  │   DataValidator(      │            │     metric="max_rel")         │ ╒
╒  │    tol=0.01,"max_rel") │            │   .validate(d_X, X_ref,       │ ╒
╒  │   .validate(          │            │     name="gemm_output")       │ ╒
╒  │    mean_gpu, mean_ref) │            │                               │ ╒
╒  │ ✅ CHECK-1b: variance  │            │ V1/V2: tol=1e-3 (тривиально)  │ ╒
```

```
│  │ ✅ CHECK-1c: std_dev   │             │ V3/V4: tol=1e-3 (GEMM точность)│ │
│  │ ✅ CHECK-1d: mean_mag  │             └────────────────────────────────┘ │
│  └──────────┬─────────────┘                                                │
│             └──────────────────────────────────────────────┘               │
│                                       ▼                                     │
│  ┌─ STEP 3 ──────────────────────────────────────────────────────────┐     │
│  │  step_3_debug_post_gemm()                                          │     │
│  │  Statistics(d_X) → CPU                                             │     │
│  │  Вход: complex64 (d_X!)  → Выход: StatisticsResult (mixed types)   │     │
│  │    .mean        : complex64   (комплексное среднее d_X)            │     │
│  │    .variance    : float32     (дисперсия |d_X|)                    │     │
│  │    .std_dev     : float32     (σ(|d_X|))                           │     │
│  │    .mean_magnitude: float32   (E[|d_X|])                           │     │
│  │                                                                    │     │
│  │  NumPy reference: stats из X_ref                                   │     │
│  │    mean_ref = X_ref.mean(axis=1)       → complex64                 │     │
│  │    var_ref  = np.abs(X_ref).var(axis=1) → float32                  │     │
│  │    std_ref  = np.abs(X_ref).std(axis=1) → float32                  │     │
│  │    mmag_ref = np.abs(X_ref).mean(axis=1) → float32                 │     │
│  │                                                                    │     │
│  │  ✅ CHECK-3a: stats(d_X) vs stats(X_ref)   tol=0.01                │     │
│  │  ✅ CHECK-3b: V1/V2 (W=I): stats(d_X) ≈ stats(d_S) — перекрёстный  │     │
│  └────────────────────────────────────────────────────────────────────┘     │
│                                       │                                     │
│                                       ▼                                     │
│  ┌─ STEP 4 ──────────────────────────────────────────────────────────┐     │
│  │  step_4_window_fft()                                               │     │
│  │  Hamming + zero-pad + FFT → d_spectrum → D2H → CPU                 │     │
│  │  Возвращает: List[complex<float>] → np.array(complex64)            │     │
│  │  shape: (n_ant × n_fft,) → reshape → (n_ant, n_fft)                │     │
│  │                                                                    │     │
│  │  NumPy reference:                                                  │     │
│  │    hamm    = np.hamming(n_samples).astype(np.float32)              │     │
│  │    padded  = np.pad(X_ref * hamm[None,:],                          │     │
│  │                    [(0,0),(0, n_fft - n_samples)]).astype(np.complex64)│ │
│  │    spec_ref = np.fft.fft(padded).astype(np.complex64)              │     │
│  │    mag_ref  = np.abs(spec_ref).astype(np.float32)                  │     │
│  │    expected_bin = round(f0 * n_fft / fs)   ← 2e6/1465 ≈ 1365       │     │
│  │                                                                    │     │
│  │  ✅ CHECK-4a: |spec_gpu| vs mag_ref                                │     │
│  │    DataValidator(tol=0.01, metric="max_rel")                       │     │
│  │  ✅ CHECK-4b: peak bin в спектре                                   │     │
│  │    peak_bin_gpu = argmax(|spec_gpu[0]|)                            │     │
│  │    DataValidator(tol=2, metric="abs")  ← допуск 2 бина             │     │
│  │    .validate(peak_bin_gpu, expected_bin, name="peak_bin")          │     │
│  │                                                                    │     │
│  │  📊 PLOT: После CHECK — рисуем спектр (обязательно для V3/V4!)     │     │
│  │    plot_spectrum(mag_ref, spec_gpu, n_fft, fs, variant, beam_id=0) │     │
│  │    Сохранить: Results/Plots/strategies/spectrum_V{n}.png           │     │
│  └────────────────────────────────────────────────────────────────────┘     │
│                                       │                                     │
│                                       ▼                                     │
│  ┌─ STEP 5 ──────────────────────────────────────────────────────────┐     │
│  │  step_5_debug_post_fft()                                           │     │
│  │  Statistics(d_magnitudes) → CPU                                    │     │
│  │  Вход: float32 (d_magnitudes = |d_spectrum|)                       │     │
│  │                                                                    │     │
│  │  NumPy reference: stats от mag_ref                                 │     │
│  │    mean_mag_ref = mag_ref.mean(axis=1)  → float32                  │     │
│  │    var_ref      = mag_ref.var(axis=1)   → float32                  │     │
│  │                                                                    │     │
│  │  ✅ CHECK-5: stats(|spectrum|)_gpu vs stats(mag_ref)   tol=0.01    │     │
│  └────────────────────────────────────────────────────────────────────┘     │
│                                       │                                     │
│                  ┌────────────────────┼────────────────────┐                │
│                  ▼                    ▼                    ▼                │
│  ┌─ STEP 6.1 ─────────┐  ┌─ STEP 6.2 ─────────┐  ┌─ STEP 6.3 ─────────────┐│
│  │ step_6_1_one_max_  │  │ step_6_2_all_      │  │ step_6_3_global_       ││
│  │ parabola()         │  │ maxima()           │  │ minmax()               ││
│  │                    │  │                    │  │                        ││
│  │ per beam float32:  │  │ per beam:          │  │ per beam float32:      ││
```

```
│  │  .beam_id (u32)    │  │  list of peaks      │  │  .min_magnitude        ││
│  │  .bin_index (u32)  │  │  (freq+mag)         │  │  .max_magnitude        ││
│  │  .magnitude (f32)  │  │                     │  │  .dynamic_range_dB     ││
│  │  .freq_offset(f32)│  │  ✅ CHECK-6.2:      │  │                         ││
│  │  .refined_freq_hz  │  │   count >= 1        │  │  ✅ CHECK-6.3a:         ││
│  │                    │  │   peak near f0      │  │   min < max            ││
│  │  ✅ CHECK-6.1:     │  └────────────────────┘  │  ✅ CHECK-6.3b:         ││
│  │   |freq - f0|      │                          │   dyn_range > 0 дБ     ││
│  │   < 50 кГц         │                          │  ✅ CHECK-6.3c: (V3/4) ││
│  │   DataValidator(   │                          │   dyn_range NumPy≈GPU  ││
│  │   tol=50e3,"abs")  │                          └────────────────────────┘│
│  └────────────────────┘                                                     │
└─────────────────────────────────────────────────────────────────────────────┘
```

📋 ТАБЛИЦА CHECK-ТОЧЕК (14 штук)

ID	Шаг	GPU данные	Reference	Тип входа	Метрика	Порог
CHECK-0	setup	shape(d_S), shape(d_W)	(n_ant,n_samples)	int	==	точно
CHECK-1a	step_1	mean[n_ant]: complex64	S_ref.mean(1)	вектор complex64	max_rel	0.01
CHECK-1b	step_1	variance[n_ant]: float32	`|S_ref|.var(1)`	вектор float32	max_rel	0.01
CHECK-1c	step_1	std_dev[n_ant]: float32	`|S_ref|.std(1)`	вектор float32	max_rel	0.01
CHECK-1d	step_1	mean_mag[n_ant]: float32	`|S_ref|.mean(1)`	вектор float32	max_rel	0.01
CHECK-2	step_2	d_X[n_ant,n_samp]: complex64	W_ref@S_ref	матрица complex64	max_rel	1e-3
CHECK-3a	step_3	stats(d_X)	stats(X_ref)	вектор mixed	max_rel	0.01
CHECK-3b	step_3 (V1/2)	stats(d_X)	stats(d_S)	вектор mixed	max_rel	0.01
CHECK-4a	step_4	`|spec_gpu|` `[n_ant,n_fft]` : float32	mag_ref	матрица float32	max_rel	0.01
CHECK-4b	step_4	peak_bin (int)	round(f0·n_fft/fs)	скаляр int	abs	2 бина
CHECK-5	step_5	stats(`|spectrum|`)	stats(mag_ref)	вектор float32	max_rel	0.01
CHECK-6.1	step_6_1	refined_freq_hz[n_ant]: float32	f0=2e6	вектор float32	abs	50 кГц
CHECK-6.2	step_6_2	count_peaks	≥ 1	скаляр int	>=	1
CHECK-6.3a	step_6_3	min_mag[n_ant]: float32	< max_mag	float32	<	—
CHECK-6.3b	step_6_3	dyn_range_dB[n_ant]: float32	> 0 дБ	float32	>	0

🏗️ АРХИТЕКТУРА КЛАССОВ (ООП/SOLID/GRASP/GoF)

Диаграмма классов

```
common/
├── runner.py
│   ├── SkipTest(Exception)             ← не pytest! свой
│   └── TestRunner                      ← Coordinator (GRASP)
│       + run(test_obj) → List[TestResult]
│       + print_summary(results)
│
├── validators.py                       ← Strategy (GoF)
│   ├── DataValidator                   ← ОДИН класс вместо 6!
│   │   + __init__(tolerance, metric)
│   │   + validate(actual, ref, name) → ValidationResult
│   └── (старые NumericValidator etc. → УДАЛИТЬ)
│
├── result.py                           ← Value Objects (GoF) — не трогаем
│   ├── ValidationResult
│   └── TestResult
│
├── test_base.py                        ← Template Method (GoF)
│   └── TestBase
│       + setUp()  — override в наследнике
│       + test_*() — override в наследнике
│       + add(ValidationResult)
│       + run() → TestResult
│
├── gpu_context.py                      ← Singleton (GoF) — не трогаем
│   └── GPUContextManager
│
└── reporters.py                        ← Observer (GoF) — не трогаем
    └── ConsoleReporter, JSONReporter

strategies/
├── numpy_reference.py                  ← Information Expert (GRASP)
│   └── NumpyReference
│       + __init__(S, W, fs, f0, n_fft)
│       + X_ref   : np.ndarray[complex64]
│       + spec_ref : np.ndarray[complex64]
│       + mag_ref  : np.ndarray[float32]
│       + stats(arr) → dict
│       + expected_peak_bin → int
│
├── signal_factory.py                   ← Factory Method (GoF) + Strategy (GoF)
│   ├── ISignalSource(ABC)              ← Strategy interface
│   │   + generate(cfg) → (d_S, S_ref, d_W, W_ref)
│   ├── GpuCwSignalSource               ← Concrete: V1/V2
│   ├── GpuCwDelayedSignalSource        ← Concrete: V3/V4
│   ├── FileSignalSource                ← Concrete: V5 (заглушка)
│   └── SignalSourceFactory
│       + create(variant: SignalVariant) → ISignalSource
│
├── pipeline_step_validator.py          ← Facade (GoF) + Template Method
│   └── PipelineStepValidator
│       + __init__(proc, ref: NumpyReference, save_to_disk=False)
│       + run_step_0(d_S, d_W) → TestResult    # CHECK-0
│       + run_step_1() → TestResult             # CHECK-1a/b/c/d
│       + run_step_2() → TestResult             # CHECK-2
│       + run_step_3() → TestResult             # CHECK-3a/b
│       + run_step_4() → TestResult             # CHECK-4a/b + PLOT
│       + run_step_5() → TestResult             # CHECK-5
│       + run_step_6_1() → TestResult           # CHECK-6.1
│       + run_step_6_2() → TestResult           # CHECK-6.2
│       + run_step_6_3() → TestResult           # CHECK-6.3a/b
│       + run_all() → TestResult                # все шаги
│
└── test_strategies_pipeline.py         ← TestBase наследник
    └── TestStrategiesPipeline(TestBase)
        + setUp()          ← выбор SignalVariant, создание proc
        + test_v1_clean()
        + test_v2_noise()
        + test_v3_phase()
```

```
        + test_v4_phase_noise()
        + test_v5_from_file()   ← заглушка
```

Принципы

Принцип	Где применён
SRP	NumpyReference отдельно от PipelineStepValidator
OCP	ISignalSource — добавить V6 без изменения остального
LSP	FileSignalSource заменяет GpuCwSignalSource везде
DIP	TestStrategiesPipeline зависит от ISignalSource, не от реализации
Strategy (GoF)	ISignalSource + DataValidator (metric)
Factory Method (GoF)	SignalSourceFactory.create(variant)
Facade (GoF)	PipelineStepValidator скрывает proc.step_N()
Template Method (GoF)	TestBase.run() + PipelineStepValidator.run_all()
Information Expert (GRASP)	NumpyReference знает свои данные и считает stats сам
Creator (GRASP)	SignalSourceFactory создаёт ISignalSource

🗂️ ФАЙЛОВАЯ СТРУКТУРА (что создаём)

Новые файлы

```
Python_test/
├── common/
│   ├── runner.py                      ← НОВЫЙ (TestRunner + SkipTest)
│   └── validators.py                  ← ПЕРЕПИСЬ (DataValidator)
│
└── strategies/
    ├── numpy_reference.py             ← НОВЫЙ (NumpyReference)
    ├── signal_factory.py              ← НОВЫЙ (ISignalSource + Factory)
    ├── pipeline_step_validator.py     ← НОВЫЙ (PipelineStepValidator)
    └── test_strategies_pipeline.py    ← НОВЫЙ (TestStrategiesPipeline)
```

Изменяемые файлы (убрать pytest)

```
Python_test/
├── conftest.py                        ← убрать import pytest
├── strategies/conftest.py             ← убрать import pytest
├── strategies/test_debug_steps.py     ← → TestBase класс
├── strategies/test_base_pipeline.py   ← → TestBase класс
├── strategies/test_strategies_step_by_step.py ← → TestBase класс
├── fft_func/*.py                      ← убрать pytest
├── statistics/*.py                    ← убрать pytest
└── (и далее по порядку...)
```

📅 ПОРЯДОК РЕАЛИЗАЦИИ

```
Сессия 1: Инфраструктура common/
  1. common/runner.py   (TestRunner + SkipTest)
  2. common/validators.py  (DataValidator — заменить все 4 старых)

Сессия 2: strategies/ — новые классы
  3. strategies/numpy_reference.py
  4. strategies/signal_factory.py     (ISignalSource + V1/V2/V3/V4 + V5-заглушка)
  5. strategies/pipeline_step_validator.py

Сессия 3: strategies/ — тесты
  6. strategies/test_strategies_pipeline.py   (5 тестов × 14 check)
  7. Убрать pytest из strategies/conftest.py + старых test_*.py

Сессия 4: Остальные модули (убрать pytest)
  fft_func → statistics → signal_generators → heterodyne →
  filters → vector_algebra → capon → range_angle
```